Backend Engineering Launchpad – Case Study

Flow: Workflow Builder

Author : Airtribe

Flow

BACKGROUND

In today's digital ecosystem, organizations depend on a variety of tools and services like CRMs, communication platforms, databases, cloud storage, and more. The ability to seamlessly connect these systems and automate workflows has become critical.

This project will have you design and implement a backend for a workflow automation platform (similar to Make or n8n) that enables users to build, manage, and monitor workflows connecting multiple services.

OBJECTIVE

Develop a reliable and extensible distributed workflow orchestration system that allows users to:

- **Design workflows** by connecting triggers and actions across different services.
- **Automate tasks** through conditional logic, branching, and error handling.
- **Monitor workflows** with logs, status tracking, and failure recovery mechanisms.

While the primary focus is backend functionality, the system must expose well-defined APIs to integrate with a potential frontend builder interface.

KEY FEATURES

Workflow Definition (Draft → Publish)

- A workflow is a list of nodes connected in order, with optional if/else branches.
- Two workflow states: Draft (editable) and Published (frozen, ready for execution).

Triggers

- Webhook trigger with a secret key.
- Schedule trigger using cron strings (e.g., */5 * * * *).

Node Types (3–4 core nodes)

- HTTP Request (GET/POST with headers and body).
- Condition (evaluate a JSON field for true/false).
- Delay (pause execution for N seconds).
- Notify (send Email or Slack via a single provider).

Workflow Execution & Runs

- Execute nodes sequentially (with support for branches).
- Persist each run's status, start/end time, and node-level logs (input/output summary, duration).

Workflow Management

- APIs to create, start, pause, resume, or delete workflows.
- Ability to reschedule workflows and view full execution history.

Conditional Logic & Branching

- Support conditions, loops, and branching paths within workflows.

Failure Handling & Recovery

- Automatic retries for failed steps.
- Error notifications to the user.

Logging & Monitoring

- Maintain detailed logs for all workflow runs.
- Provide real-time monitoring of workflow status.
- Track overall system health.

Authentication & Limits

- Basic multi-user authentication.
- Guardrails for fair usage and security.

TECHNICAL REQUIREMENTS

- Implement core functionality as RESTful APIs.
- Use a reliable database to store workflow definitions, execution states, and logs.
- Implement authentication and authorization for workflow creation and management.
- Provide a modular design for easily adding new connectors.
- Ensure scalability to handle multiple concurrent workflow executions.
- Design for fault tolerance and idempotency to guarantee consistent execution.

ASSESSMENT CRITERIA

- **Functionality:** Does the system work as intended? Does it meet all the requirements stated above?

- **Code Quality:** Is your code clean, organized, well-commented, and following best practices?

- **Design and Structure:** Is the system well-designed? Does it demonstrate a good understanding of system design principles and patterns?

- **Documentation:** Is your report comprehensive and clear? Does it effectively explain the choices made and how to use the project?

- **Presentation:** Do you effectively demonstrate and explain your system and the decisions made during its development?

DELIVERABLES

1. The final, functional product.
2. README file outlining how to use the system, API documentation, the design decisions and other necessary information.
3. Public link of the Github repository
4. Explainer video demonstrating your project work